

System & Software Security

Condensed Study Notes

1. MALWARE

Definition: Software that infects systems *without user consent* to steal data (bank details, passwords, emails), disrupt operations, or alter system behaviour. Can exfiltrate info, corrupt/delete files, impair availability or integrity.

1.1 Entry Vectors & Mitigations

- **Phishing Emails** — Verify sender, do not click links
- **Malicious Websites** — Keep browser & plugins updated
- **Unpatched Vulnerabilities** — Apply patches, use Web Application Firewall (WAF)
- **Infected Removable Media** — Disable autorun, scan devices on insertion

General mitigation: keep antivirus up-to-date; exercise caution with links and attachments.

1.2 Types of Malware

- **Virus** — Malicious code attached to executable files; spreads when infected file is shared; ranges from harmless to deleting/modifying data; once active, infects other programs.
- **Worm** — Self-replicates across a network; attaches to files; does NOT need a host program; slows networks; spreads very rapidly after infecting a host.
- **Trojan Horse** — Disguised as legitimate software (e.g. an online game); unlike viruses, binds to non-executable files such as images or audio files.
- **Ransomware** — Encrypts victim's data with unknown key; demands ransom payment to decrypt; victim cannot resume use until amount is paid.
- **Adware** — Displays unwanted ads and pop-ups; bundled with free software downloads; generates revenue for the distributor.
- **Spyware** — Silently collects private information (passwords, browsing habits) and sends it to a third-party hacker.
- **Logic Bomb** — Dormant malware activated by a trigger event; can destroy hardware components (fans, hard drives, power supplies) by overdriving them.
- **Rootkit** — Modifies the OS to create a backdoor; attacker uses backdoor for remote access; exploits software vulnerabilities to modify system files.
- **Backdoor** — Bypasses normal authentication; grants persistent future access even after the original vulnerability is patched.
- **Keylogger** — Records every keystroke to capture passwords and sensitive information; transmits data to the attacker.
- **Botnet** — Turns infected PC into a 'zombie' node; controlled remotely as part of a larger bot network for coordinated attacks.

1.3 Anti-Malware Strategies

- **Proactive Prevention** — regularly update software and OS to patch vulnerabilities
- **Defense-in-Depth** — multiple security layers: firewalls, IDS, web filtering
- **Zero Trust Architecture** — strict least-privilege access to restrict malware movement
- **User Training** — educate users to avoid suspicious links and email attachments
- **Data Backups** — regular offline backups (critical for ransomware recovery)
- **Network Segmentation** — divide networks into isolated zones to contain breaches

1.4 Anti-Malware Tools

- **Antivirus / Anti-Malware Software** — signature-based detection & real-time scanning (Malwarebytes, Microsoft Defender)

- EDR (Endpoint Detection & Response) — advanced monitoring, detecting, and responding to threats at endpoint level
- Network Security Tools — firewalls and traffic analysers such as Wireshark and Fiddler to detect unusual traffic
- Email & Web Filtering — block malicious websites and phishing emails at the gateway
- Deception Technology — honeypots and decoys to lure and analyse attacker behaviour
- Memory Protection Tools — defend against fileless malware that operates entirely in RAM

2. UNINTENTIONAL (NON-MALICIOUS) PROGRAMMING FLAWS

Accidental bugs, logic errors, or oversights during design and coding that create security vulnerabilities — no malicious intent, but can cause data breaches, system instability, or arbitrary code execution. Common examples: buffer overflows, improper input validation, race conditions.

2.1 Buffer Overflow

Occurs when a program writes more data to a storage buffer than it can hold, overwriting adjacent memory. Results in crashes, corrupted software, or arbitrary code execution. Remains a serious and well-known threat.

- **Stack-based Overflow:** Stack has fixed size set by developer. If user input exceeds allocated space and no bounds check exists, the stack overflows. Harmless alone but dangerous with malicious input.
- **Heap-based Overflow:** Targets data in the dynamically allocated heap memory pool.
- **Integer Overflow:** Arithmetic operation produces a value too large for the integer type; can indirectly trigger a buffer overflow.

2.2 Incomplete Mediation (Input Validation Errors)

Failure to consistently enforce security policies across *all* entry points. Attacker manipulates input that the system accepts without full validation, leading to unauthorized access or data corruption.

- URL tampering example: `?item=1&price=100` manipulated to `?item=1&price=1`
- SQL Injection / XSS via unvalidated form or URL fields
- Session hijacking when session is not re-validated after login

Prevention Strategies:

- Consistent input validation at every entry point across the application
- Strict access controls — use a reference monitor to govern every requested access
- Regular security audits, module testing, and integration testing

2.3 TOCTOU — Time-of-Check to Time-of-Use (Race Condition)

A race condition where a resource's state changes *between* the security check and the actual use of that resource, enabling unauthorized access, privilege escalation, or data corruption.

- **Vulnerability Window:** The small critical time interval between check and use.
- **Classic File System Example:** Privileged program checks `/home/user/Log` is owned by the user (safe) → attacker deletes it and creates a symlink to `/etc/passwd` → program overwrites `/etc/passwd` instead.
- **Common in:** File system operations (Unix), local sockets, database transactions.
- **Mitigation:** Make check+use atomic; use file descriptors instead of file names so the resource cannot be swapped between check and use.

2.4 Other Key Programming Flaws

- **Unterminated Null-Terminated String** — Character array intended as a string is missing the `\0` terminator. Consequences: buffer overflows, segmentation faults, garbage output from adjacent memory, and security vulnerabilities.
- **Memory Leaks / Uninitialised Variables** — Application fails to release unused memory or uses variables before assigning values. Can lead to system instability and unpredictable behaviour.
- **Parameter Length Issues** — No minimum/maximum byte limits enforced on URL parameters or form fields. Enables buffer overflows and SQL injection. Mitigation: WAF rules, password complexity policies.

- **Undocumented Access Points (Rogue APs / Hidden Interfaces)** — Unauthorised Wi-Fi access points or hidden manufacturer 'chicken bit' interfaces. Create open doors for attackers to bypass perimeter defences, move laterally, or establish persistence.
- **Unsafe Utility Programs** — Trojan-horse utilities (fake antivirus, rogue registry cleaners); rogue optimisers that scare users; free tools bundled with adware or spyware.
- **Security Misconfiguration** — Default passwords unchanged, unnecessary services left enabled, overly permissive access rights. A very common and easily exploited vulnerability.
- **Integer Overflow / Underflow** — Calculation exceeds max/min integer value causing unexpected behaviour in loops or memory allocation.

3. COUNTERMEASURES

3.1 For Users

- Use software acquired from reliable, reputable sources only
- Test new software in an isolated environment before production use
- Open attachments only when you know they are safe
- Treat every website as potentially harmful
- Create and maintain regular backups of important data

3.2 For Developers — Software Engineering Techniques

- Information Hiding — describe what a module does, not how (abstraction)
- Modularity — well-defined coupling and cohesion between components
- Mutual Suspicion — components validate and distrust each other's input
- Confinement — limit blast radius; damage in one component cannot spread
- Simplicity — complexity is the enemy of security
- Genetic Diversity — varied implementations reduce single-attack susceptibility
- Testing — unit, integration, function, performance, acceptance, installation, regression, penetration

3.3 Design Principles for Security (Saltzer & Schroeder)

- **Least Privilege** — Grant only the minimum permissions necessary for the task
- **Economy of Mechanism** — Keep security design simple and small
- **Open Design** — Security must not depend on secrecy of design or implementation
- **Complete Mediation** — Every access to every object must be checked against access policy
- **Permission Based (Default Deny)** — Default is to deny; access must be explicitly granted
- **Separation of Privilege** — Require multiple independent conditions to grant access
- **Least Common Mechanism** — Minimise shared resources between different users/processes
- **Ease of Use** — Security mechanisms must be usable, or users will work around them

3.4 Additional Security Countermeasures

- Penetration Testing — actively test the system to find vulnerabilities before attackers do
- Proofs of Program Correctness — formal mathematical verification of code correctness
- Validation — rigorously verify that software meets security requirements
- Defensive Programming — anticipate what could go wrong and code against it; do not assume correct input
- Trustworthy Computing Initiative — all developers undergo security training; secure dev practices instituted company-wide

3.5 Countermeasures That Do NOT Work

- **Penetrate-and-Patch** — Hurried, misses the broader context of the fault, and focuses on one failure rather than the complete system. Does not address root causes.
- **Security by Obscurity** — The belief that a system is secure as long as its internal mechanisms are kept secret. Fails the moment internals are discovered or leaked.

4. SECURE SOFTWARE DEVELOPMENT LIFECYCLE (SSDLC)

SSDLC embeds security at *every phase* of software development rather than treating it as a late-stage add-on. Traditional SDLC addresses security mainly during testing; SSDLC integrates it from planning through maintenance.

4.1 Key Principles of Secure SDLC

- **Security by Design** — Integrate security requirements from the very first planning and design stages.
- **Continuous Monitoring** — Ongoing security checks and issue resolution throughout every development phase.
- **Risk Assessment** — Identify, prioritise, and mitigate vulnerabilities early — before they become costly.
- **Education & Training** — All developers receive comprehensive security training for effective issue handling.
- **Collaboration** — Dev, Ops, and Security teams share knowledge and coordinate toward common security goals.

4.2 Phases of SSDLC

- **1. Planning** — Define security requirements; identify risks; create a security plan. Security is baked in from the start.
- **2. Design** — Put security plan into action; make design decisions to handle threats; conduct threat modelling and design review.
- **3. Implementation** — Write code using secure coding practices; perform code reviews and static analysis to catch security issues.
- **4. Testing** — Penetration testing, vulnerability scanning, security code review; verify the software handles security threats correctly.
- **5. Deployment** — Ensure the release process is secure; perform security assessment and secure configuration before go-live.
- **6. Maintenance** — Continuous monitoring; regular updates to address new threats; ongoing lifecycle protection after release.

4.3 SDLC vs SSDLC Comparison

Aspect	SDLC	SSDLC
Focus	Delivering functional software	Security at every stage
Objective	Meet business/functional requirements	Protect against vulnerabilities
Security timing	Mainly testing phase	Design through deployment
Testing	Separate security phase	Integrated in each phase
Extra phases	Standard phases only	Risk Assessment, Threat Modelling, Static Analysis, Secure Config
Documentation	Functionality & features	Security measures, threat models, risk assessments
Speed	Faster delivery	Slightly longer due to security steps
Adaptability	More adaptable to changes	More effort to change due to security focus

5. PRIVACY BY DESIGN (PbD)

An approach requiring privacy to be *embedded from the design stage* and throughout the development lifecycle — not an afterthought. Privacy becomes an integral part of core functionality.

7 Principles of Privacy by Design

- **1. Proactive not Reactive; Preventative not Remedial**
 - A privacy-first attitude. Actively build processes and procedures to prevent privacy risks before they occur, rather than reacting after a breach or complaint.
- **2. Privacy as the Default Setting**

– Automatic maximum privacy protection — users need not take any action. Includes: collection limitation (only legally allowed data), data minimisation (only what is necessary), use/retention/disclosure limitation (only for stated purpose), and security (encryption, technical & org measures).

- **3. Privacy Embedded into Design**

– Privacy is a core component woven into every design decision, not bolted on at the end. Must be seamless — neither obvious nor awkward — while maintaining full functionality.

- **4. Full Functionality — Positive-Sum, not Zero-Sum**

– Reject false trade-offs between privacy and usability/security. Both can be achieved simultaneously. Positive-sum mindset = innovate to integrate privacy seamlessly; zero-sum mindset = treat privacy as a sacrifice.

- **5. End-to-End Security — Full Lifecycle Protection**

– Data is protected from initial collection through secure destruction, and everything in between. Ensures confidentiality (secret), integrity (unmodified), and availability throughout the processing lifecycle.

- **6. Visibility & Transparency — Keep it Open**

– Clearly and consistently document and communicate privacy policies and procedures. Builds accountability and trust. Supported by accessible complaint submission/resolution and independent verification.

- **7. Respect for User Privacy — Keep it User-Centric**

– Always keep users' privacy interests at the forefront. Put control in users' hands to manage their own data. Every design decision reflects respect for user privacy; actively seek user engagement.

QUICK REFERENCE — KEY TERMS

Term	Definition
Buffer Overflow	Writing past a buffer's boundary, corrupting adjacent memory
TOCTOU	Race condition: resource state changes between security check and use
Incomplete Mediation	Security checks not enforced consistently at all entry points
Virus	Malicious code attached to executables; spreads via file sharing
Worm	Self-replicating malware that spreads without a host program
Trojan Horse	Malware disguised as legitimate software; binds to non-executables
Ransomware	Encrypts victim data; demands ransom payment for decryption key
Rootkit	Modifies OS to create hidden backdoor for remote access
Logic Bomb	Dormant malware triggered by a specific event
Keylogger	Records keystrokes to capture passwords and sensitive data
SSDLC	Security embedded in every phase of the development lifecycle
PbD	Privacy by Design — privacy baked into design from the start
Least Privilege	Grant only minimum access rights needed for the task
Defense-in-Depth	Multiple overlapping security layers
Zero Trust	Never trust, always verify; strict least-privilege access
Penetrate-and-Patch	INEFFECTIVE: hurried fix of one flaw; misses systemic issues
Security by Obscurity	INEFFECTIVE: hiding internal mechanisms does not equal security